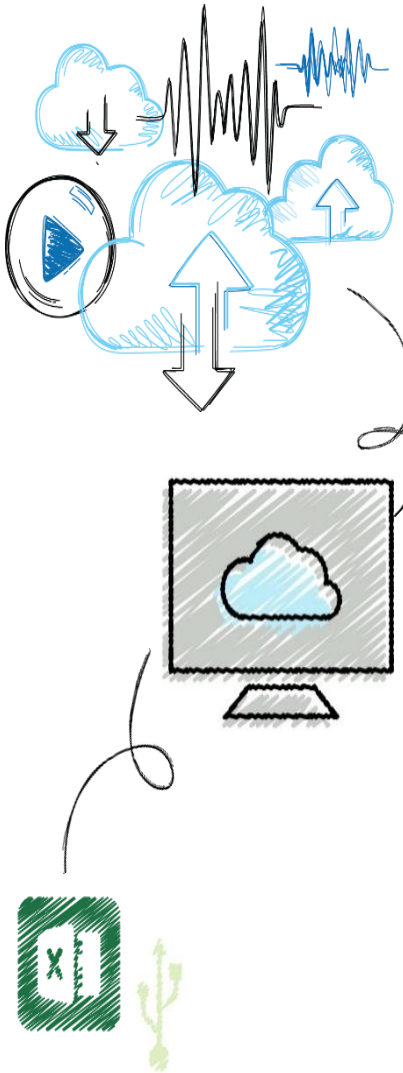
ICT703 Programming

Module 2 – Topic 2

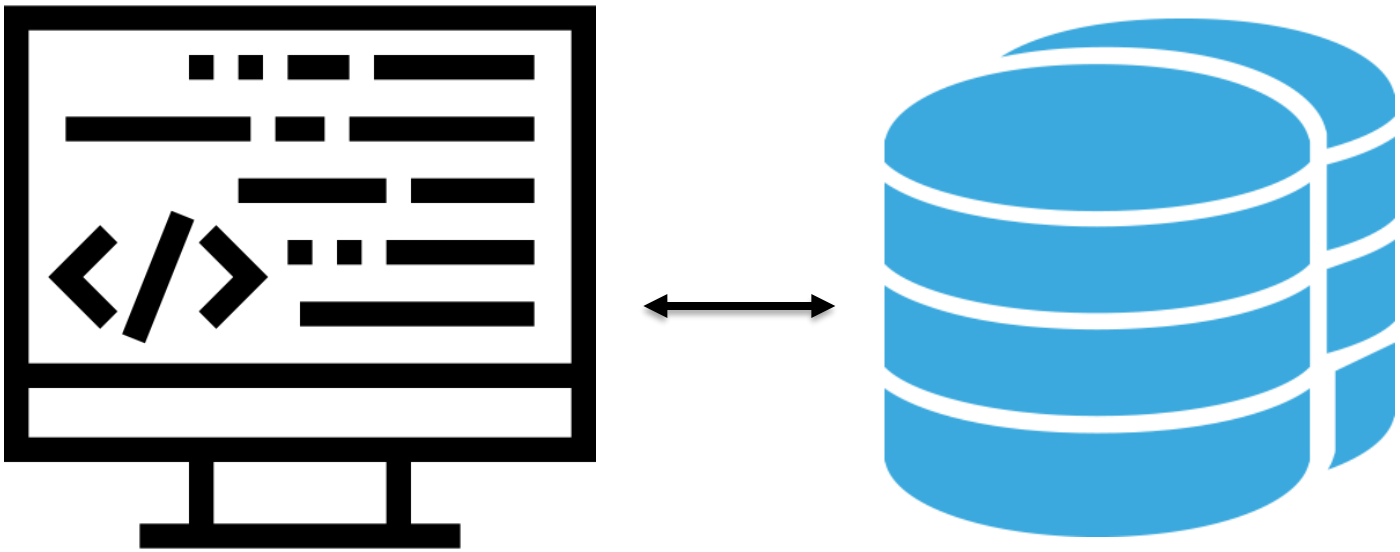
Python Data Types



Objectives

1. Investigate the importance of data in programs
2. Use strings for the terminal input and output of text
3. Use integers and floating point numbers in arithmetic operations
4. Construct arithmetic expressions
5. Initialize and use variables with appropriate names

Data and Programs



Programs

Data

Data

- Can be internal to a program
- Can be external – files, databases, cloud, etc.
- Rule of thumb – separate data from your program
  - Easier to program
  - Easier to update/maintain

Data Types in Python

- Strings (Text) – “Hello Everyone”
- Integers (Whole Numbers) – 23, 45
- Real Numbers (Float) – 3.1415, 100.9
- Boolean – True or False
- Lists – [1, 2, 3, 4] or [“cat”, ”dog”, “bird”]
- Sets - {"apple", "banana", "cherry"}
- Dictionaries – { "brand": “Kia", "model": “Cerato", "year": 2020}
- And more …

Data Types (1 of 2)

- A **data type** consists of a set of values and a set of operations that can be performed on those values
- A **literal** is the way a value of a data type looks to a programmer
- **int** and **float** are **numeric data types**
  - They represent numbers

Data Types (2 of 2)

Type of Data	Python Type Name	Example Literals
Integers	int	−1, 0, 1, 2
Real numbers	float	−0.55, .3333, 3.14, 6.0
Character strings	str	“Hi”, “”, ‘A’, “66”

String Literals (1 of 2)

- In Python, a string literal is a sequence of characters enclosed in single or double quotation marks
- **“** and **””** represent the **empty string**
- Double-quoted strings are handy for composing strings that contain single quotation marks or apostrophes:

print(“I’m using a single quote in this string!”)

I’m using a single quote in this string!

String Literals (2 of 2)

- Use “” and ”””for multi-line paragraphs

print(“””This very long sentence extends

all the way to the next line.”””)

This very long sentence extends all the way to the next line

String Concatenation

- You can join two or more strings to form a new string using the concatenation operator **+**

Print(“Hi “ + “there,” + “Ken!”)

‘Hi there, Ken!

Variables and the Assignment Statement (1 of 3)

- A **variable** associates a name with a value
  - Makes it easy to remember and use later in program
- Variable naming rules:
  - Reserved words cannot be used as variable names
    - Examples: **if**, **def**, and **import**
  - Name must begin with a letter or **_**
  - Name can contain any number of letters, digits, or _
  - Names are case sensitive
    - Example: **WEIGHT** is different from **weight**
  - Tip: begin each word in the variable name with an uppercase, except the first one (Example: **interestRate**)

Variables and the Assignment Statement (2 of 3)

- Programmers use all uppercase letters for **symbolic constants** (contain values that the program never changes)
  - Examples: **TAX_RATE** and **STANDARD_DEDUCTION**
- Variables receive initial values and can be reset to new values with an **assignment statement**

  <variable name> = <expression>
  - Subsequent uses of the variable name in expressions are known as **variable references**
- Example:

  firstName = “Ken”

  secondName = “Lambert”

  fullName = firstName + “ ” + secondName

  Print(fullName)

  ‘Ken Lambert’

Variables and the Assignment Statement (3 of 3)

- Variables serve two purposes:
  - Help the programmer keep track of data that change over time
  - Allow the programmer to refer to a complex piece of information with a simple name (called abstraction)

Numeric Data Types and Character Sets

- The first applications of computers were to crunch numbers
- The use of numbers in many applications is still very important

Integers

- In real life, the range of **integers** is infinite
- A computer's memory places a limit on the magnitude of the largest positive and negative integers
  - Python's **int** typical range:

    -21^{31} to $2^{31} - 1$
- Integer literals are written without commas

Floating-Point Numbers (1 of 2)

- Real numbers have **infinite precision**
  - The digits in the fractional part can continue forever
- Python uses **floating-point** numbers to represent real numbers
- Python's **float** typical range:

  -10^{308} to 10^{308} and
- Typical precision: 16 digits

Floating-Point Numbers (2 of 2)

- A floating-point number can be written using either ordinary **decimal notation** or **scientific notation**

Decimal Notation	Scientific Notation	Meaning
3.78	3.78e0	3.78×10^{0}
37.8	3.78e1	3.78×10^{1}
3780.0	3.78e3	3.78×10^{3}
0.378	3.78e−1	3.78×10^{-1}
0.00378	3.78e−3	3.78×10^{-3}

Expressions

- A literal evaluates to itself
- A variable reference evaluates to the variable's current value
- **Expressions** provide easy way to perform operations on data values to produce other values
- When entered in a Python program:
  - an expression's operands are evaluated
  - its operator is then applied to these values to compute the value of the expression

- Operator vs Operand
  - An **operator** is the 'function' that performs the operation
  - The **operand** is the input to that function.
  - In the expression 3 + 4 = 7, the **operator** is '+' - since it's telling us how to perform the operation - and the **operands** are 3 and 4 - the inputs upon which the operation is acting

Arithmetic Expressions (1 of 3)

- An **arithmetic expression** consists of operands and operators combined in a manner that is already familiar to you from learning algebra

Operator	Meaning	Syntax
–	Negation	–a
**	Exponentiation	a ** b
*	Multiplication	a * b
/	Division	a / b
//	Quotient	a // b
%	Remainder or modulus	a % b
+	Addition	a + b
–	Subtraction	a – b

Arithmetic Expressions (2 of 3)

- **Precedence rules**:
  - ** has the highest precedence and is evaluated first
  - Unary negation is evaluated next
  - *, /, and % are evaluated before + and −
  - + and − are evaluated before =
  - With two exceptions, operations of equal precedence are **left associative**, so they are evaluated from left to right
    - ** and = are **right associative**
  - You can use () to change the order of evaluation

Arithmetic Expressions (3 of 3)

- When both operands of an expression are of the same numeric type, the resulting value is also of that type
- When each operand is of a different type, the resulting value is of the more general type
  - Example: **3 / 4** is **0**, whereas **3 / 4.0** is **.75**

Mixed-Mode Arithmetic and Type Conversions (1 of 4)

- **Mixed-mode arithmetic** involves integers and floating-point numbers:
  - 3.14 * 3 ** 2
  - 28.26
- You must use a **type conversion function** when working with *input* of numbers
  - It is a function with the same name as the data type to which it converts
- Input function returns a string as its value
  - You must use the **int** or **float** function to convert the string to a number before performing arithmetic

Mixed-Mode Arithmetic and Type Conversions (2 of 4)

Conversion Function	Example Use	Value Returned
int(<a number or a string>)	int(3.77) int("33")	3 33
float(<a number or a string>)	float(22)	22.0
str(<any value>)	str(99)	'99'

Mixed-Mode Arithmetic and Type Conversions (3 of 4)

- Note that the **int** function converts a **float** to an **int** by truncation, not by rounding

Print(int(6.75))

6

Print(round(6.75))

7

Mixed-Mode Arithmetic and Type Conversions (4 of 4)

- Type conversion also occurs in the construction of strings from numbers and other strings

```
profit = 1000.55
print('$' + profit)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'float' objects
```

- Solution: use **str** function

```
print('$' + str(profit))
$1000.55
```

- Python is a strongly **typed** programming language